

LABO ALU

Départements : TIC

Unité d'enseignement CSN

Auteurs : **Jonathan Nicolet**
Romain Hurni

Professeur : **Etienne Messerli**
Assistant : **Bruno Sutterlet**

Classe : **CSN-A**
Salle de labo : **A07**

Date : **25.03.2026**

1 Introduction

On doit faire un ALU en VHDL avec différents calculs choisis par un opcode.
Pour ce faire, on va d'abord s'intéresser aux différentes possibilités de faire chaque opération.

On va ensuite déterminer laquelle choisir pour que le système soit le plus optimisé possible en limitant la taille des composants et en essayant de les réutiliser pour différentes fonctions.

Avec ceci nous pourrons créer un schéma, puis l'implémenter en VHDL.

Il ne nous restera plus qu'à tester avec Questasim puis sur une carte.

2 Analyse et conception

On veut trouver la façon la plus optimisée d'effectuer chaque opération de l'ALU, pour se faire il faut déjà lister les différentes implémentations possibles.

2.1 Addition

Il suffit d'utiliser un additionneur. On peut utiliser l'additionneur `add4_full` déjà implémenté et l'adapter en `addn_full`.

2.2 Multiplication

Pour réaliser la multiplication par 2 il faut décaler notre opérande d'un bit vers la gauche. Il suffit donc de relier les bits `[N-1 .. 0]` du signal d'entrée aux bits `[N .. 1]` du signal de sortie et mettre le poids faible de la sortie à '0'.

On peut aussi additionner le nombre avec lui-même.

2.3 ET logique

On peut réaliser cela grâce à l'opérande AND

2.4 Incrémentation

Elle peut être réalisée avec un additionneur où la deuxième valeur vaut 1.

2.5 Soustraction ($A - B$)

On peut créer un soustracteur à la manière de l'additionneur en changeant juste l'opération.

Une autre solution est d'utiliser notre additionneur et de réaliser le complément à deux (cà2) de B, la valeur à soustraire.

2.6 Soustraction ($B - A$)

On peut utiliser la même solution que pour l'autre soustraction, ou reprendre directement la valeur de celle-ci et l'inverser (car $B - A = -(A - B)$)

2.7 OU logique

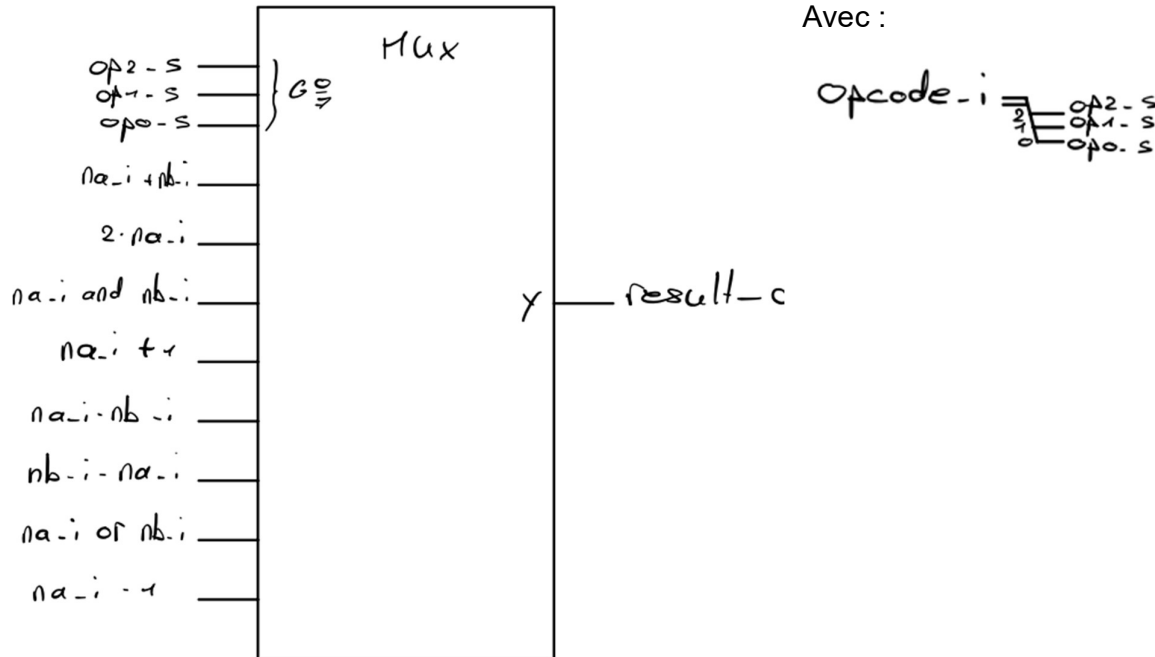
Comme pour ET il suffit d'utiliser l'opérateur OR

2.8 Décrémentation

Similaire à la soustraction, on peut soit implémenter un soustracteur soit additionner le complément à deux de 1 (qu'on peut calculer au préalable ou sur le moment).

3 Réalisation et implantation

Une solution est de prendre un mux a 3 bits (8 entrées) et de relier chaque opérations.



Cependant on peut optimiser cela :

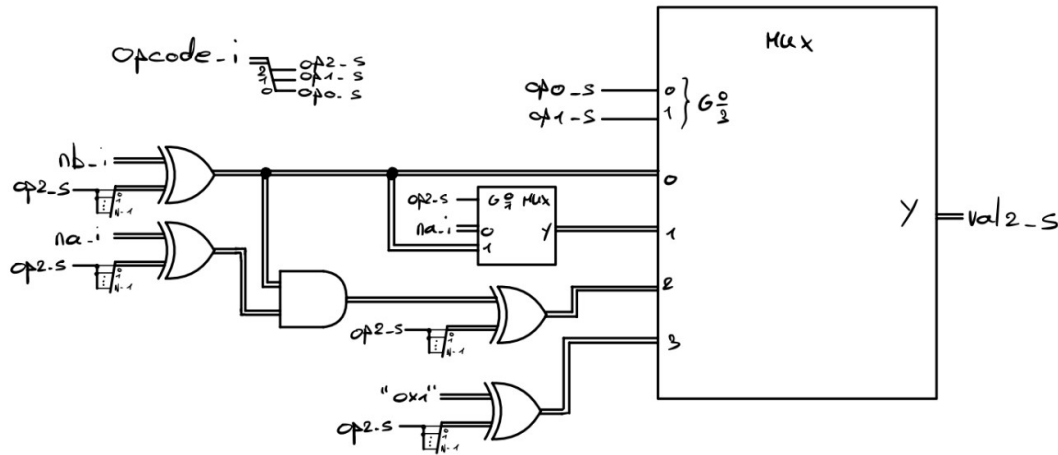
- Quand le bit de poids fort est à '1', on effectue une soustraction (à part pour le OR)
- Si on utilise $B - A = -(A - B)$, alors le premier opérande est toujours na_i (sauf pour Or et AND) -> on peut utiliser un mux pour déterminer le deuxième opérande. Dans ce cas on fait deux fois le cà2, les +1 de ceux-ci s'annule, il suffit donc de laisser le cin à 0 dans ce cas.
- On peut aussi passer le résultat du OR et AND dans l'additionneur et l'additionner à 0.

Notes pour le OR et le AND :

Grace à Morgan, il est possible d'utiliser uniquement une porte AND pour effectuer le AND et le OR, simplement en choisissant si entrées et sortie de la porte AND sont inversé ou non (en utilisant op2).

$$\rightarrow \neg(A * B) = \neg A + \neg B = A + B$$

On doit donc déterminer les deux opérandes de l'addition.

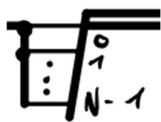


Deuxième opérande :

Rappel :

opcode_i	Opération	Valeur sur result_o
000	Addition	$na_i + nb_i$
001	Multiplication	$2 \cdot na_i$
010	ET logique	$na_i \text{ and } nb_i$
011	Incrémentation	$na_i + 1$
100	Soustraction	$na_i - nb_i$
101	Soustraction	$nb_i - na_i$
110	OU logique	$na_i \text{ or } nb_i$
111	Décrémentation	$na_i - 1$

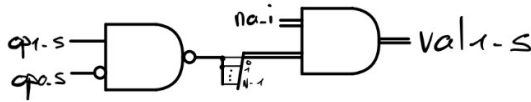
Affectation d'un bit sur un bus de N bits :



On a donc comme entrée dans le MUX des valeurs dépendant de op2_s:

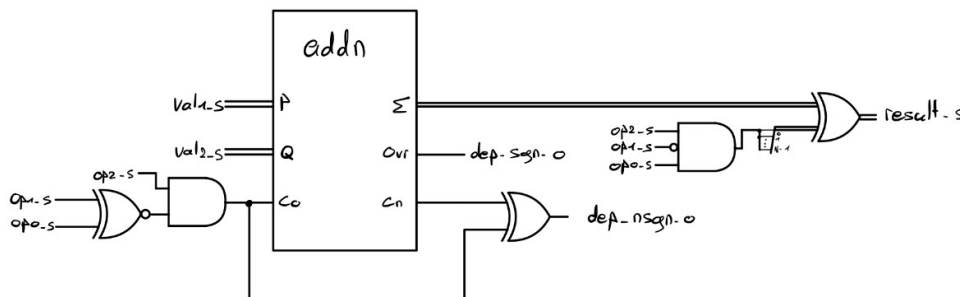
- On fait soit une addition soit une soustraction, le bit op2_s nous dit donc si on active ou non le cà2 (inversion des bits + carry de l'additionneur à '1', voir ensuite)
- On a soit na_i (pour l'additionner à lui-même et donc le multiplier par 2) soit le cà2 de nb_i (car $B-A = -(A-B)$).
- Avec op2_s = 0 on fait un AND. Sinon on fait $\text{not}((\text{not } na_i) \text{ AND } (\text{not } nb_i))$ ce qui nous fait $na_i \text{ OR } nb_i$.
- Si op2_s est vrai on à 1, sinon -1 (son cà2)

Première opérande :



Le deuxième opérande est na_i sauf quand on fait OR ou AND (quand opcode(1..0) = 10). Dans ce cas elle est de 0 afin d'additionner 0 au résultat de la porte logique.

Addition :

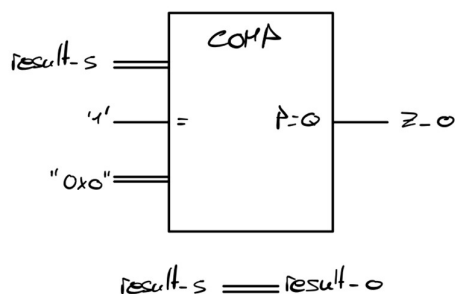


On a un cin pour le complément à deux, donc lorsque op2_s est a '1'. A l'exception de B-A car on fait le cà2 avant et après l'addition ce qui revient à faire $-(+1) + 1$. On ne fait donc rien à la place afin de simplifier le circuit. Lorsqu'on fait B-A on doit également inverser bits du résultat pour refaire le cà2.

Un dépassement de nombre signé est déterminé par l'overflow et un de nombre non signé par la carry out à 1 pour des additions et par la carry out à 0 pour des soustractions. On reprend donc le signal de carry in pour déterminer si on inverse la carry out. Ceci fonctionne car B-A est traduit en deux soustractions successives $-(A-B)$, le dépassement est donc donné par $\text{not}(\text{not}(\text{carry out})) = \text{carry out}$.

Par ailleurs une opération logique (OR et AND) ne retournera pas de dépassement car on les additionne à 0 et qu'elles ne sont pas considérées comme une soustraction -> carry out n'est pas inversé.

Résultat :



Il nous reste finalement à tester si le résultat vaut 0 et à set result_o

On a un signal intermédiaire result_s car on doit le relire pour le tester à 0 et qu'on ne peut pas lire un signal de sortie par convention.

4 Réponse question

Expliquer comment vous pouvez obtenir à l'aide de votre ALU, le résultat des tests $na_i < nb_i$ ou $na_i > nb_i$:

Il faut faire l'opération $na_i - nb_i$ et regarder le bit de signe. Cependant sa valeur est fausse s'il y a eu un overflow, on doit donc inverser le résultat dans ce cas avec un xor, on a donc :

$na_i < nb_i$ si $result_o(N-1) \text{ xor } dep_sgn_o$

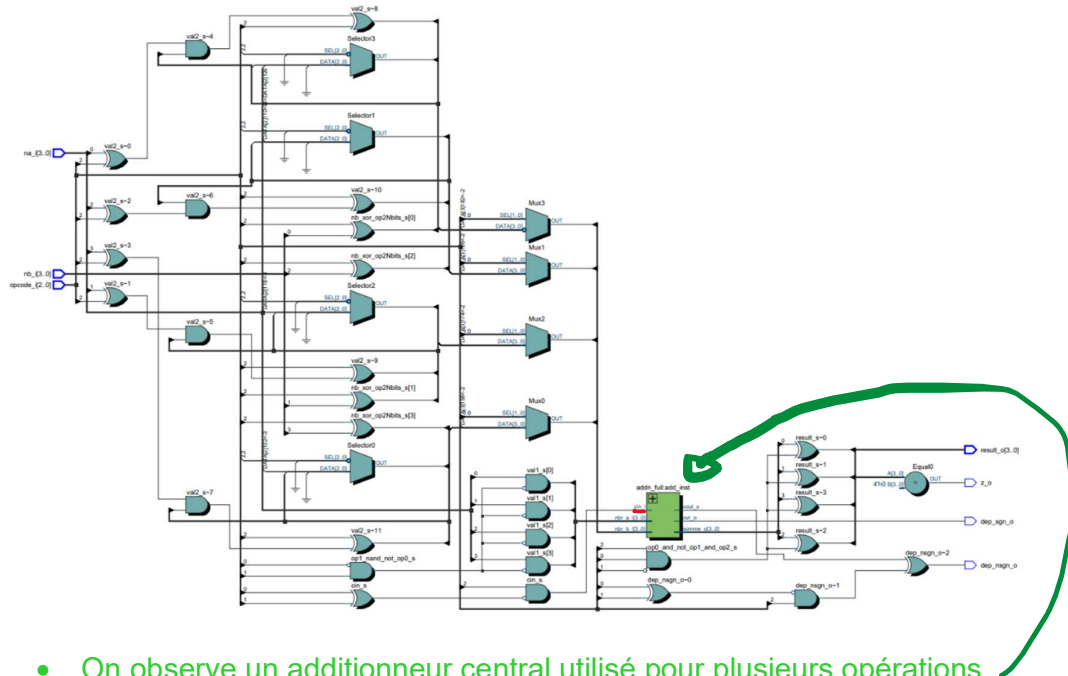
Pour $na_i > nb_i$ il faut tester en plus si le résultat n'est pas égal à 0 (car l'inverse de $na_i < nb_i$ est $na_i \geq nb_i$). On a donc :

$na_i > nb_i$ si $(\text{not } (result_o(N-1) \text{ xor } dep_sgn_o)) \text{ and } (\text{not } z_o)$

Ce qui peut être simplifié en $\text{not } ((result_o(N-1) \text{ xor } dep_sgn_o) \text{ or } z_o)$

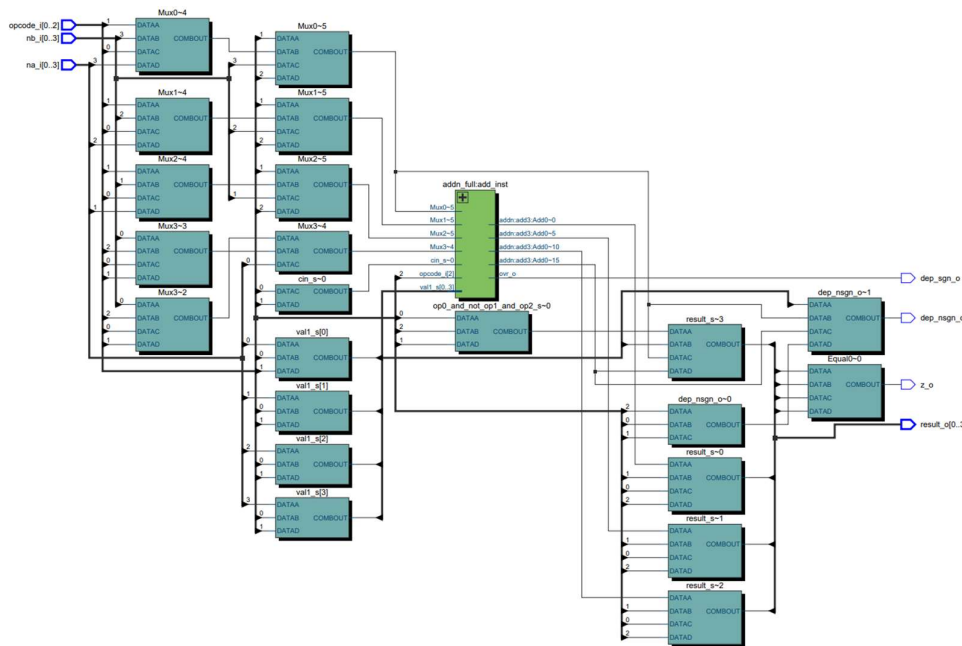
5 Simulation

Vue RTL :



- On observe un additionneur central utilisé pour plusieurs opérations
- Les multiplexeurs permettent de sélectionner les opérandes selon opcode_i
- Les opérations logiques (AND/OR) sont intégrées en amont de l'additionneur
- Le signal cin permet d'activer le complément à deux

Vue technologique :



- L'ALU est implémenté avec des LUTs* et registres du FPGA
- L'additionneur est mappé sur des chaînes de carry (carry chain)
- Le nombre de ressources dépend de N
- Les multiplexeurs sont implémentés avec des LUTs

* LUTs = Look-Up Table (petit block mémoire dans un FPGA qui sert à implémenter des fonctions logiques)

Nombres d'éléments en fonctions de bits :

N = 4 bits -> 28 cellules logiques

Compilation Hierarchy Node		Logic Cells
[-] alu_nbits_top	1	28 (22)
[-] addn_fulladd_inst	1	6 (1)
[-] addn:add3	1	5 (5)

N = 8 bits -> 52 cellules logiques

Compilation Hierarchy Node		Logic Cells
[-] alu_nbits_top	1	52 (42)
[-] addn_fulladd_inst	1	10 (1)
[-] addn:add3	1	9 (9)

N = 12 bits -> 76 cellules logiques

Compilation Hierarchy Node		Logic Cells
[-] alu_nbits_top	1	76 (62)
[-] addn_fulladd_inst	1	14 (1)
[-] addn:add3	1	13 (13)

Résultats test Bench

```
# End time: 01:33:25 on Mar 23,2026, Elapsed time: 0:03:36
# Errors: 0, Warnings: 0
# vsim -voptargs="+acc" work.alu_top_tb -GVAL_N=6
# Start time: 01:33:25 on Mar 23,2026
# ** Note: (vsim-8009) Loading existing optimized design _opt1
# Loading std.standard
# Loading std.textio(body)
# Loading ieee.std_logic_1164(body)
# Loading ieee.numeric_std(body)
# Loading ieee.math_real(body)
# Loading work.logger_pkg(body)
# Loading work.project_logger_pkg
# Loading work.common_pkg(body)
# Loading work.alu_top_tb(test_bench)#1
# Loading work.alu_nbits_top(struct)#1
# Loading work.addn_full(struct)#1
# Loading work.addn(flot_don)#1
# Loading work.addn(flot_don)#2
# 1
# ** Note: >> [NOTE] : @ 1 ns Start of simulation
# Time: 1 ns Iteration: 0 Instance: /alu_top_tb
# ** Note: >> [NOTE] : @ 500466 ns
# +-----+
# | FINAL REPORT |
# +-----+
# | Nb warnings = 0
# | Nb errors = 0
# |
# | Verbosity level is : note
# |
# | *** VOUS ETES LES MEILLEURS ***
# | *** Bravo, pas d'erreurs ***
# |
# | END OF SIMULATION
# Time: 500466 ns Iteration: 0 Instance: /alu_top_tb
```

Le testbench permet de valider le bon fonctionnement de l'ALU en appliquant différentes combinaisons de valeurs aux entrées na_i, nb_i et opcode_i.

L'ensemble des opérations définies par l'ALU a été testé :

- addition
- multiplication par 2
- opérations logiques (AND, OR)
- incrémentation et décrémentation
- soustractions (A – B et B – A)

Des cas particuliers ont également été vérifiés afin de garantir la robustesse du circuit :

- détection d'un résultat nul (z_o)
- dépassement de capacité pour nombres signés (dep_sgn_o)
- dépassement pour nombres non signés (dep_nsgn_o)

Les résultats observés en simulation correspondent aux valeurs attendues pour chaque opération.

Les signaux de sortie réagissent correctement en fonction des entrées, ce qui valide le bon fonctionnement de l'ALU.

Run -all :

Contrairement aux précédents laboratoires, il n'y a pas besoin d'effectuer manuellement le « run -all » car il est déjà écrit dans le fichier run_alu_top_tb.tcl. On peut le voir en dernière ligne.

run_alu_top_tb.tc

```
# files compilation
do ../comp_alu.tcl

#Chargement fichier pour la simulation
if {$argc>0} {
    vsim -voptargs="+acc" work.alu_top_tb -GVAL_N=$1
} else {
    vsim -voptargs="+acc" work.alu_top_tb -GVAL_N=6
    ...
}

configure wave -timelineunits ns
update
WaveRestoreZoom {0 ns} {509385 ns}

run -all
```

Explications parties principales du code alu_nbits.vhdl :

```
op2_s <= opcode_i(2);  
op1_s <= opcode_i(1);  
op0_s <= opcode_i(0);
```

Le signal opcode_i est décomposé en trois bits (op2_s, op1_s, op0_s) afin de simplifier la logique de contrôle.

Cette séparation permet d'implémenter plus facilement les conditions logiques utilisées pour générer les opérandes et les signaux de contrôle.

```
op1_nand_not_op0_s <= op1_s nand not(op0_s);  
val1_s <= na_i and (val1_s'range => op1_nand_not_op0_s);
```

Le premier opérande (val1_s) est dérivé de na_i.

Le signal op1_nand_not_op0_s permet de contrôler si cet opérande doit être actif ou forcé à zéro.

Cette logique permet notamment de « désactiver » la fonction de l'additionneur lors des opérations logiques, où le calcul est réalisé autrement. (L'additionneur devient comme un fil en faisant + 0).

```
op2_Nbits_s <= (val2_s'range => op2_s);  
nb_xor_op2Nbits_s <= nb_i xor op2_Nbits_s;
```

Le bit op2_s est étendu sur N bits afin de pouvoir être utilisé dans des opérations vectorielles.

Le XOR avec nb_i permet d'inverser conditionnellement les bits de nb_i.

Cette technique est utilisée pour implémenter le complément à deux dans les opérations de soustraction ou pour transformer le and en or.

```

with op2_s select
    mux_mult_ou_soustr_s <= na_i          when '0',
                                nb_xor_op2Nbits_s when '1',
                                (others => '0')  when others;
    
```

Ce multiplexeur permet de sélectionner le second opérande en fonction de op2_s.

Si op2_s = 0, on utilise na_i (cas de la multiplication par 2)

Si op2_s = 1, on utilise nb_i éventuellement inversé (cas des soustractions)

Cela permet de mutualiser les ressources en utilisant un seul chemin de données.

```

with opcode_i(1 downto 0) select
    val2_s <= nb_xor_op2Nbits_s          when "00",
    mux_mult_ou_soustr_s                  when "01",
    ((na_i xor op2_Nbits_s) and nb_xor_op2Nbits_s) xor op2_Nbits_s when "10",
    std_logic_vector(to_unsigned(1, N)) xor op2_Nbits_s          when "11",
    (others => '0')                  when others;
    
```

Le second opérande (val2_s) est généré à l'aide d'un multiplexeur contrôlé par les deux bits de poids faible de l'opcode.

Chaque cas correspond à une opération :

"00" : utilisation de nb_i (ou son complément à deux)

"01" : multiplication ou soustraction selon op2_s

"10" : opérations logiques AND/OR via transformations booléennes

"11" : incrémentation ou décrémentation via ajout de ± 1

Cette approche permet d'implémenter toutes les opérations avec un seul additionneur.

```

cin_s <= (op1_s xnor op0_s) and op2_s;
    
```

Le signal cin_s permet d'ajouter 1 lors du calcul du complément à deux.

Il est activé uniquement pour certaines opérations de soustraction, en fonction de l'opcode.

Cette logique évite d'ajouter systématiquement une constante, optimisant ainsi le circuit.

```
add_inst : entity work.addn_full
```

L'additionneur est le composant central de l'ALU.

Il est utilisé pour réaliser :

- les additions
- les soustractions (via complément à deux)
- les incrémentations et décrémentations

Cette réutilisation permet de minimiser le nombre de composants logiques.

```
dep_sgn_o <= ovr_s;  
dep_nsgn_o <= cout_s xor (op2_s and (op1_s xnor op0_s));
```

Deux types de dépassements sont gérés :

dep_sgn_o : correspond au dépassement signé (overflow), directement issu de l'additionneur.

dep_nsgn_o : correspond au dépassement non signé, basé sur la retenue (cout_s).

Une correction est appliquée pour les opérations de soustraction afin de garantir un comportement correct.

```
op0_and_not_op1_and_op2_s <= op0_s and not(op1_s) and op2_s;  
result_s <= res_add_s xor (res_add_s'range => op0_and_not_op1_and_op2_s);
```

Dans le cas de l'opération B - A, le résultat est obtenu en inversant le résultat de A - B.

Cette inversion est réalisée à l'aide d'un XOR conditionnel appliqué sur tous les bits.

Cette technique permet d'éviter un second passage dans l'additionneur.

```
z_o <= '1' when (result_s = (result_s'range => '0')) else '0';
```

Le signal z_o est activé lorsque le résultat est nul.

Cela permet d'identifier facilement certaines conditions, notamment dans des comparaisons.

```
result_o <= result_s;
```

Le résultat final est directement assigné à la sortie après traitement interne.

L'utilisation d'un signal intermédiaire (result_s) permet de faciliter les traitements supplémentaires, comme la détection de zéro.

Conclusion

Nous avons réussi à optimiser le nombre de composants utilisés comme souhaités notamment en réutilisant l'additionneur pour toutes les opérations et en effectuant différentes transformations logiques.

Les tests en simulation et sur carte nous ont permis de valider le bon fonctionnement de notre ALU.

Date : 25.03.2026

Noms des étudiants : Jonathan Nicolet, Romain Hurni